

UF5

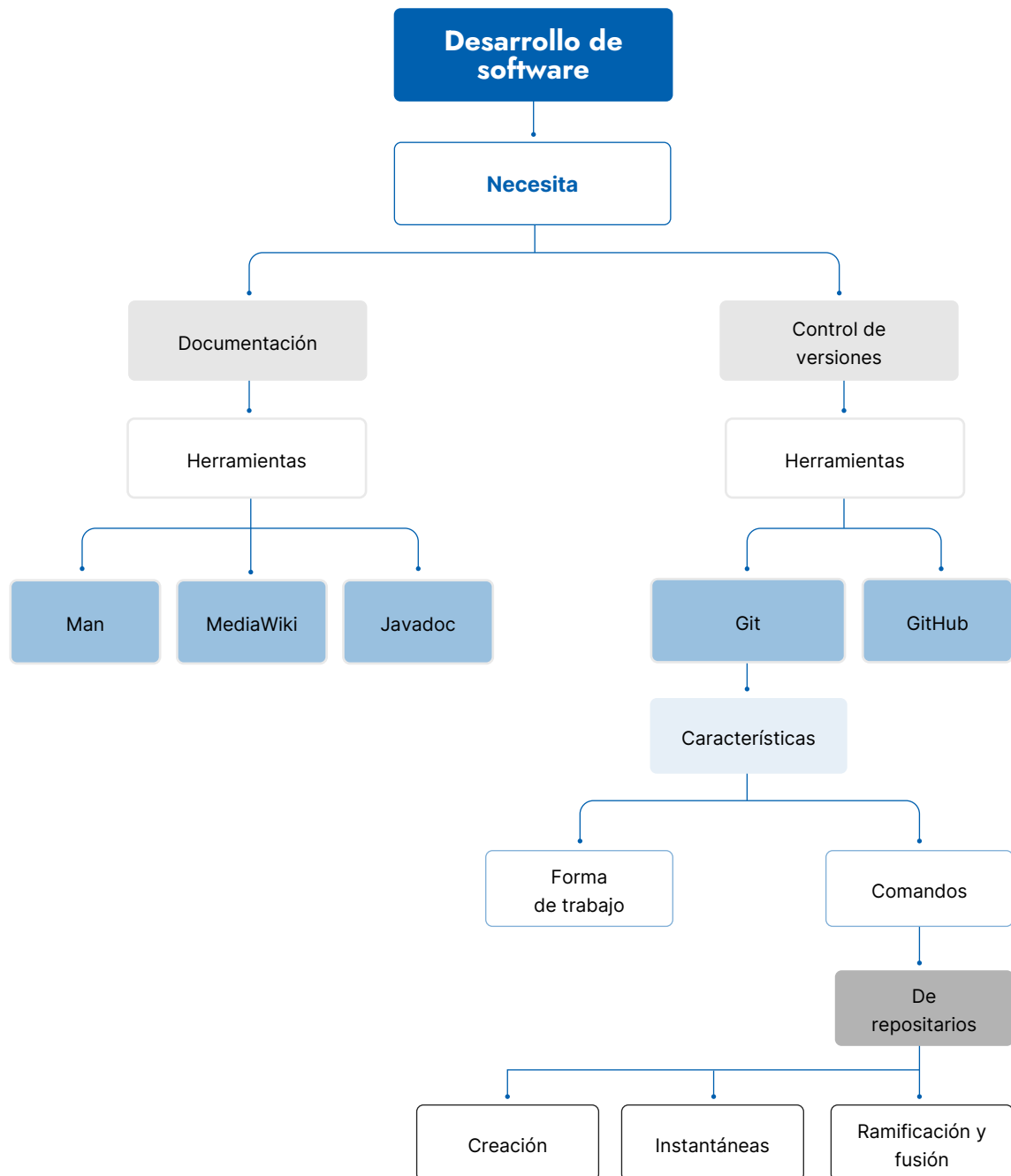
Documentación y control de versiones

Despliegue de aplicaciones web

ÍNDICE

Mapa conceptual	03
1. Introducción	04
1.1. Documentación del software.....	04
1.2. Control de versiones.....	06
2. Documentación de software	10
2.1. Man	10
2.2. MediaWiki	12
2.3. Javadoc	15
3. Control de versiones	22
3.1. Git	23
A. Forma de trabajo.....	24
B. Comandos.....	24
3.2. GitHub.....	27
A. Cuentas de GitHub	29

Mapa conceptual



01 Introducción

Desde que el software es concebido y hasta que deja de usarse, pasa por una serie de procesos que determinan aspectos tan importantes como el desarrollo, el mantenimiento y el uso. A esta serie de procesos o fases se las conoce como **ciclo de vida del software**, que ayudará a desarrollar programas de mejor calidad y de una manera más eficiente.

Existen muchos modelos que representan de distintas maneras las fases y el orden del ciclo de vida del software, como los modelos en cascada, en espiral o de prototipos, pero la mayoría tienen en común los siguientes aspectos:

- »recogida y análisis de los requerimientos de usuario;
- »especificación de lo que debe hacer el programa;
- »implementación o desarrollo del programa;
- »comprobación del buen funcionamiento;
- »despliegue del programa y explotación en un entorno real;
- »mantenimiento, que incluye los cambios necesarios para adaptarlo a los nuevos requerimientos o para corregir errores.

Para cada uno de estos aspectos, se genera una **documentación** que recogerá la información relevante. Por ejemplo, un manual de instalación para el despliegue del programa, un manual de usuario para su explotación o la documentación del diseño interno que facilita el desarrollo y el mantenimiento.

Por otro lado, tanto en la fase de implementación como en la de mantenimiento, existirán distintas copias del programa, por lo que se debe llevar un **control de versiones** minucioso para evitar problemas como la pérdida de cambios realizados en las versiones más recientes o el uso de versiones antiguas del programa.

Tanto el proceso de documentación como el control de versiones, dos aspectos muy importantes en el ciclo de vida del software, se van a desarrollar a lo largo de esta unidad.

1.1. Documentación del software

Es difícil encontrar un proyecto de software que no adquiera cierto tamaño y complejidad, por lo que es necesario establecer un mecanismo de documentación de los distintos aspectos que conforman las fases del ciclo de vida del software. Así, no importa si el proyecto lo desarrolla una única persona o un gran equipo interdisciplinar, todos los procesos de un proyecto de software deben documentarse de acuerdo con una estructura y unas normas establecidas de antemano.

Tener un proyecto con una **documentación accesible y de calidad** ayuda a evitar ciertos problemas. A continuación, se ven algunos ejemplos:

- » Un documento consensuado con el cliente donde se recoja cuáles son los requerimientos puede evitar posibles malentendidos a la hora de determinar si el producto final cumple o no las funciones pactadas.
- » Si un proyecto se debe modificar (por ejemplo, por un cambio de moneda de curso legal en un país), es mucho más fácil planificar y realizar los cambios si se dispone de la documentación interna dirigida a los desarrolladores.
- » Cuando parte o todo el equipo humano de un proyecto de software lo abandona y debe reemplazarse, el nuevo equipo lo tendrá más fácil si se dispone de la documentación.
- » Un buen manual de usuario o de administración permitirá que las personas que lo usen sean más autónomas, remitan menos consultas y aumente el grado de satisfacción por poder sacarle mayor partido a la aplicación.

Entonces, si parece que todo son ventajas, ¿por qué es tan habitual encontrar software sin documentar, sobre todo, si es hecho a medida? Son muchas las razones por las que se deja de documentar, pero podemos destacar las siguientes:

- » la falta de tiempo por plazos de entrega muy ajustados derivados de una mala planificación;
- » la carencia de una cultura de empresa que le otorgue a la documentación la importancia que merece;
- » a los desarrolladores les parece que la documentación es un trabajo tedioso y poco atractivo.

Después de haber visto por qué debe documentarse un software y, en muchas ocasiones, por qué no se hace, quizá es hora de plantearse qué se debería **tener en cuenta a la hora de redactar la documentación**. Estas son unas indicaciones generales:

- » entendible y, por lo tanto, útil para ese colectivo. No es lo mismo escribir para quien pueda tener la necesidad de modificar el código que hacerlo para quien va a usar el programa.
- » Es muy importante estructurar bien el documento e incluir un índice que permita localizar rápidamente la información que se necesita en cada momento.
- » Hay que tener definido un estándar, a ser posible, con plantillas definidas, para conseguir una documentación homogénea.
- » Debe escribirse una buena introducción que permita al lector saber qué es lo que va a encontrar en ese documento.
- » Hay que incluir ejemplos contextualizados en la documentación técnica que aclaren las explicaciones e indicaciones dadas.

- » Es importante acompañar el texto de esquemas y capturas de pantalla que indiquen los pasos que hay que seguir para realizar ciertos procesos, sobre todo, si el documento está dirigido al usuario final de la aplicación.
- » Utilizar un formato de archivo estándar y que sea fácilmente mantenible.
- » Indicar claramente la fecha de edición de la documentación y la versión del programa a la que hace referencia. Esto evitará pérdidas de tiempo derivadas de seguir unas indicaciones de una versión diferente a la usada.
- » Solo hay que documentar lo que realmente es necesario y hay que revisar la vigencia de los documentos para adaptarlos o, simplemente, retirarlos.

Por último, cuando esté creada la documentación, es muy importante que sea fácilmente accesible por las personas que la puedan necesitar y, además, que los usuarios puedan participar en su actualización.

1.2. Control de versiones

A lo largo de la vida útil de un programa, se realizan muchas modificaciones. Algunas de ellas son corrección de errores que se detectan durante la utilización del software por parte de los usuarios y otras son mejoras o ampliaciones para adaptar el programa a las nuevas necesidades del mercado o de los propios clientes.

Si llevar un control de estas tareas de modificación a veces resulta complejo incluso en una aplicación pequeña desarrollada por un único programador, el trabajo se complica cuando la aplicación que hay que modificar o ampliar es de un tamaño mayor y en su programación participan varios desarrolladores.

Cualquier buen programador crea copias de los archivos que van a modificarse antes de efectuar cambios por si fuera necesario volver al estado original del programa. Se trata de una tarea relativamente fácil si él es el único involucrado en el proyecto, pero, cuando la aplicación se desarrolla o mantiene entre un equipo de varios programadores, la tarea se complica, y todavía más si cada programador trabaja de forma remota desde su casa, situación que cada vez es más común.

Como puede deducirse de estas situaciones que acaban de describirse, llevar un control de cambios es imprescindible para que el proyecto se desarrolle de una forma ordenada y sin incidentes. Saber en todo momento en qué versión se trabaja y qué desarrollador realizó las modificaciones es crucial para poder volver de forma inmediata a una versión anterior en caso de detectar que la última no funciona como se espera.

Llevar a cabo un control de versiones de forma optimizada va más allá de hacer copias de seguridad de todo o parte del proyecto cuando se tiene que abordar alguna modificación. El uso de un sistema de control de versiones mientras se desarrolla o mantiene el software de una aplicación tiene las siguientes ventajas:

- » facilita poder volver a estados anteriores del proyecto rápidamente, ya que todos los cambios, tanto en la creación y eliminación de archivos como en las modificaciones, quedan registrados;
- » favorece que los equipos de trabajo se concentren en su labor de programar y, por tanto, sean más eficientes, lo que se traduce en un menor tiempo de desarrollo;
- » permite la trazabilidad de los cambios. Poder consultar los estados por los que ha pasado un fragmento de código puede ser muy útil para que los programadores puedan comprender su funcionalidad y por qué se decidió programar de esa manera, sobre todo, si proviene de versiones muy antiguas.

1

CASO PRÁCTICO

**INSTALACIÓN DE GIT Y PRUEBAS DE FUNCIONAMIENTO**

En este caso práctico, vas a instalar Git, crear un repositorio y hacer unas pruebas de funcionamiento. Puedes hacerlo en tu propia máquina real o usar una máquina virtual, como, por ejemplo, Ubuntu Desktop. Sigue los pasos que se describen a continuación:

1. Instala Git y configura tu nombre de usuario y tu correo electrónico.
2. Crea el subdirectorio *proyecto* dentro de tu directorio de usuario.
3. Crea un repositorio dentro del subdirectorio *proyecto* y utiliza el comando *status* para ver la información del repositorio.

Crea un archivo y, luego, pásalo del directorio de trabajo al área de preparación y, por último, al repositorio.

SOLUCIÓN**Instala y configura GIT**

Los siguientes pasos se han realizado en el terminal de Ubuntu Desktop.

1. Para instalar Git y configurar tu usuario y correo electrónico, ejecuta las siguientes instrucciones:

```
admin01@cliente:~$ sudo apt install git
admin01@cliente:~$ git config --global user.name pol
admin01@cliente:~$ git config --global user.email pol@pruebas.local
```

Aclaración: Si el nombre lleva algún espacio en blanco, hay que ponerlo entre comillas dobles, por ejemplo: **git config --global user.name "Pol Green"**.

2. Antes de crear el directorio, asegúrate de que estás en tu directorio de usuario:

```
admin01@cliente:~$ cd ~
admin01@cliente:~$ mkdir proyecto
```

3. Para crear el repositorio dentro del directorio *proyecto*, ejecuta estos comandos en el terminal:

```
admin01@cliente:~$ cd proyecto
admin01@cliente:~/proyecto$ git init
ayuda: Usando 'master' como el nombre de la rama inicial. Este nombre de rama
predeterminado
ayuda: está sujeto a cambios. Para configurar el nombre de la rama inicial para usar
en todos
ayuda: de sus nuevos repositorios, que suprimirán esta advertencia, llame a:
ayuda:
ayuda: git config --global init.defaultBranch <nombre>
ayuda:
ayuda: Los nombres comúnmente elegidos en lugar de 'maestro' son 'principal', 'troncal' y
ayuda: 'desarrollo'. Se puede cambiar el nombre de la rama recién creada mediante
este comando:
ayuda:
ayuda: git branch -m <nombre>
```

Inicializado repositorio Git vacío en /home/admin01/proyecto/.git/

Para comprobar que se ha creado el repositorio, que es un directorio oculto llamado *.git*, ejecuta el comando:

```
admin01@cliente:~/proyecto$ ls -a
. .. .git
```

Usa el comando *status* para ver la información del repositorio:

```
admin01@cliente:~/proyecto$ git status
```

En la rama master

No hay commits todavía

no hay nada para confirmar (crea/copia archivos y usa "git add" para hacerles seguimiento)

Como puedes comprobar, tu repositorio está vacío, pero listo para usarse.

4. Para este ejemplo se ha creado el archivo *index.html*.

```
admin01@cliente:~/proyecto$ nano index.html
```

Al consultar el estado de tu repositorio deben aparecer las siguientes líneas:

```
admin01@cliente:~/proyecto$ git status
```

En la rama master

No hay commits todavía

Archivos sin seguimiento:

(usa «git add <archivo>...” para incluirlo a lo que se será confirmado)

index.html

no hay nada agregado al commit pero hay archivos sin seguimiento presentes (usa “**git add**” para hacerles seguimiento)

Para pasar el archivo del directorio de trabajo al área de preparación debes usar el comando *add*:

```
admin01@cliente:~/proyecto$ git add index.html
```

Al consultar de nuevo el estado de tu repositorio, debe aparecer:

```
admin01@cliente:~/proyecto$ git status
```

En la rama master

No hay commits todavía

Cambios a ser confirmados:

(usa «git rm --cached <archivo>...” para sacar del área de stage)

nuevo archivo: index.html

Para incluir el archivo en el repositorio debes utilizar el comando *commit* con el parámetro *-m* y el mensaje que identifica la versión:

```
admin01@cliente:~/proyecto$ git commit -m v1
[master (commit-raíz) 5be4b82] v1
1 file changed, 1 insertion(+)
create mode 100644 index.html
```

Al consultar de nuevo el estado de tu repositorio debe aparecer:

```
admin01@cliente:~/proyecto$ git status
```

En la rama master

nada para hacer commit, el árbol de trabajo está limpio

Con esto ya tienes la primera versión de tu proyecto. El histórico de versiones se puede consultar con el comando *log*:

```
admin01@cliente:~/proyecto$ git log
commit 5be4b820060e722a953a1a7b9279ac8433801f5d (HEAD -> master)
Author: pol <pol@pruebas.local>
Date: Sun Jul 24 11:31:17 2022 +0200

v1
```

02 Documentación de software

En el apartado 1 de esta unidad, se han visto, de manera general, los siguientes aspectos de la documentación: su utilidad, por qué es menos frecuente de lo que cabría esperar y qué hay que tener en cuenta a la hora de su redacción. Este apartado se centra en algunas de las herramientas de documentación más utilizadas, tanto internas como externas.

2.1. Man

Es la aplicación más usada de línea de comandos que proporciona una interfaz al usuario para **acceder a los manuales de referencia** de sistemas como BSD, Unix, Mac OS o GNU/Linux. Esta aplicación se basa en otros programas para paginar y formatear la salida. Los manuales de referencia están clasificados en secciones numeradas, normalmente, del 1 al 8:

1. Programas ejecutables y órdenes del *shell*.
2. Llamadas al sistema (funciones del núcleo).
3. Llamadas a biblioteca (funciones de bibliotecas).
4. Archivos especiales, normalmente, de configuración.
5. Formatos de archivo y convenios.
6. Juegos.
7. Miscelánea (incluidos paquetes de macros y convenios).
8. Órdenes de administración del sistema (normalmente, solo para *root*).

Así, cuando se invoca a **man**, este busca la ayuda solicitada en orden en todas las secciones y subsecciones. Como después de localizar la ayuda solicitada ya no busca en más secciones, si se quiere mostrar la información de otra sección, habrá que indicarlo manualmente.

La sintaxis básica de esta aplicación es **man [opciones de man] [[sección] página]**, por ejemplo:

COMANDO	EXPLICACIÓN
<code>\$ man passwd</code>	Muestra la ayuda sobre el comando passwd , que se encuentra en la sección 1.
<code>\$ man 5 passwd</code>	Muestra el formato del archivo /etc/passwd , que se encuentra en la sección 5.
<code>\$ man -a passwd</code>	Muestra sucesivamente la ayuda localizada en cada una de las secciones que coinciden con passwd .
<code>\$ man -f passwd</code>	Imprime las descripciones cortas de passwd encontradas en todas las secciones.
<code>\$ man -k passwd</code>	Imprime las descripciones cortas de todas las entradas que coincidan con la cadena "passwd", independientemente de la sección en la que se encuentren.
<code>\$ man man</code>	Muestra la ayuda sobre el comando man , de la sección 1.

Tabla 1

Las páginas del manual se almacenan en las localizaciones indicadas en el archivo **/etc/manpath.config**, y, normalmente, pueden contener las siguientes **secciones**: NOMBRE, SINOPSIS, CONFIGURACIÓN, DESCRIPCIÓN, OPCIONES, ESTADO DE SALIDA, VALOR DEVUELTO, ERRORES, ENTORNO, FICHEROS, VERSIONES, CONFORME A, NOTAS, DEFECTOS, EJEMPLO, AUTORES y VÉASE TAMBIÉN.

La sección SINOPSIS muestra la sintaxis del comando, por ejemplo, del propio **man**:

MAN(1) Utilidades del paginador del manual MAN(1)

NOMBRE

man - interfaz de los manuales de referencia del sistema

SINOPSIS

```
man [opciones de man] [[sección] página ...] ...
man -k [opciones de apropos] regexp ...
man -K [opciones de man] [sección] term ...
man -f [whatis opciones] página ...
man -l [opciones de man] fichero ...
man -w|-W [opciones de man] página ..
```

La SINOPSIS debe interpretarse de la siguiente manera:

- » puede usarse cualquiera de las líneas que la componen sin intercambiar elementos entre ellos;
- » el texto en negrita debe teclearse tal cual, sin modificaciones;
- » los corchetes indican que su contenido es opcional;
- » el carácter "|" separa elementos que no pueden usarse a la vez;
- » los tres puntos detrás de un elemento indican que puede repetirse;
- » las cursivas o subrayados indican que el texto tiene que sustituirse por un argumento apropiado para cada momento.

Como alternativas a la aplicación man, se pueden usar otros programas locales, como info, servir en formato HTML las páginas del propio sistema a través de un servidor HTTP o acceder a uno de los muchos sitios de Internet que dan acceso a las páginas del manual en formato HTML.

2.2. MediaWiki

Esta aplicación web ofrece una wiki en la que puede desarrollarse la **documentación** de un proyecto de software de manera **colaborativa y centralizada**. Se usa por todo el mundo y el portal más importante basado en él es Wikipedia.

MediaWiki ofrece una plataforma libre y de código abierto que permite tener una documentación multilenguaje, actualizable y muy fiable. Además, gracias a un servidor web, se puede actualizar y consultar desde cualquier equipo con acceso al servidor.

El **contenido** de un sitio implementado con MediaWiki puede mantenerlo cualquier usuario o solo los permitidos en la política de grupos y usuarios. Cuando un contenido se actualiza, gracias al sistema de control de versiones, se guarda el estado anterior por si tiene que recuperarse o consultarse. Además, esta información se almacena en una base de datos cuando se trata de texto, pero, cuando se trata de imágenes u otro contenido multimedia, se almacena en el sistema de archivos.

Para mostrárselo al usuario, la información, el contenido multimedia y el de la base de datos se alojan en **páginas** —también llamadas artículos—, que, a su vez, se agrupan por tipos dentro de los **espacios de nombres**. A excepción de las páginas del espacio de nombres principal, que carece de prefijo, para identificar una página se nombra *Prefijo del espacio:Tema de la página*; por ejemplo, *Ayuda:Plantillas*.

Las diferentes páginas que componen la wiki están escritas en formato **wikitexto**, un lenguaje de marcas basado en HTML y diseñado especialmente para este tipo de aplicaciones. Wikitexto no está definido ni estandarizado, sino que cada aplicación de creación de wikis puede introducir sus propias modificaciones. Por otro lado, MediaWiki también cuenta con un editor, similar al de otros procesadores de texto, llamado *Editor Visual*. Esta herramienta es de tipo WYSIWYG (*What you see is what you get*), por lo que permite ver el resultado final mientras se edita. En la siguiente tabla se puede observar un extracto de su sintaxis:

FORMATO	EXPLICACIÓN
== Título 1 == === Título 2 === ==== Título 3 ==== ===== Título 4 =====	Encabezados de diferentes niveles.
"Texto" '''Texto''' """"Texto""""	Texto en cursiva. Texto en negrita. Texto en negrita y cursiva.
[[Página]] [[Página Título]]	Enlace a una página interna, es decir, de la misma wiki. Enlace con nombre alternativo a una página interna.
URL [URL Título]	Enlace externo. Enlace externo con nombre alternativo.
* Texto ** Texto # Texto ## Texto	Elemento de una lista de primer nivel no numerada. Elemento de una lista de segundo nivel no numerada. Elemento de una lista de primer nivel numerada. Elemento de una lista de segundo nivel numerada.
:Texto ::Texto	Párrafo sangrado de primer nivel. Párrafo con un sangrado de segundo nivel.
—	Línea horizontal.
~~~   ~~~~	Firma con un enlace a la página del usuario con su nombre.   Firma con la fecha.
[[Image:imagen]]   [[Image:imagen thumb]]	Inserta una imagen.   Inserta una miniatura.
{{subst:Plantilla}}   {{Plantilla}}	Incluye una plantilla por sustitución.   Incluye una plantilla por referencia.

Tabla 2

Además de todos los formatos propios de wikitexto, también es posible utilizar algunas etiquetas HTML para formatear el contenido de las páginas de MediaWiki. En los enlaces de esta unidad se puede encontrar la página *Ayuda:HTML en el wikitexto*, de Wikipedia, donde se recoge la lista completa.

Las **plantillas** son un tipo especial de páginas pensadas para ser insertadas en otras páginas de la wiki. Este contenido puede insertarse de dos maneras:

» **Por sustitución:** en este caso, se hace una copia del contenido actual de la plantilla y se desvincula de esta, de manera que, si el contenido de la plantilla se actualiza, la información de la página no varía.

» **Por referencia:** significa que se incluye el contenido de la plantilla en la página dinámicamente, es decir, cada vez que se accede a la página, se busca el contenido de la plantilla y se copia. Así, si cambia el contenido de la plantilla, también se refleja en la página.

Lo normal es que, al incorporar una plantilla en una página, se quiera adaptar la información al resto del contenido de la página. Por ejemplo, si se añade una plantilla que incluye una tabla con la información sobre una persona, debería aparecer su información personal nombre, apellidos, etc. Para conseguir esto, cuando se incluye la plantilla, se añaden los **parámetros** (nombre y valor) de acuerdo con esta sintaxis:

```
{{Plantilla|parámetro1=valor1|parámetro2=valor2|parámetro3=valor3...}}
```

A continuación, se puede ver parte del código de wikitexto que incluye la plantilla *Ficha de persona*, parametrizada en el artículo de Wikipedia dedicado a Linus Torvalds. Se puede obtener en la pestaña *Editar* de dicho artículo.

```
{{Ficha de persona
| imagen = LinuxCon Europe Linus Torvalds 03 (cropped).jpg
| descripción = Torvalds en 2014
| hijos = 3
| nombre de nacimiento = Torvalds Benedict Linus
| fecha de nacimiento = {{fecha|28|12|1969|e}}<ref name="nacimiento" />
| lugar de nacimiento = [[Helsinki]], [[Finlandia]]
| fecha de fallecimiento = no
| lugar de fallecimiento =
| residencia = {{bandera|Estados Unidos}} [[Portland (Oregón)|Portland]],
[[Oregón]]
| nacionalidad = [[Finlandia|finés]] [[estadounidense]]<ref
name="citLinus"/>
| ...

}}
```

En la imagen se puede ver cómo se representa la plantilla en la página de Linus Torvalds. Además se incluye un ejemplo de la misma plantilla, pero parametrizada con los datos personales de otra persona.

Linus Torvalds	Richard Stallman
	
Torvalds en 2014	Richard Stallman en 2010.
Información personal	Información personal
<b>Nombre de nacimiento</b> Torvalds Benedict Linus	<b>Nombre de nacimiento</b> Richard Matthew Stallman
<b>Nacimiento</b> 28 de diciembre de 1969 (52 años) ¹ Helsinki, Finlandia	<b>Nacimiento</b> 16 de marzo de 1953 (69 años) Manhattan (Estados Unidos)
<b>Residencia</b> Portland, Oregón	<b>Residencia</b> Boston
<b>Nacionalidad</b> finés estadounidense ²	<b>Nacionalidad</b> Estadounidense
<b>Etnia</b> finlandés sueco-parlante	<b>Religión</b> Ateísmo
<b>Religión</b> Agnosticismo	<b>Lengua materna</b> Inglés
<b>Lengua materna</b> Sueco de Finlandia	

Por último, antes de instalar MediaWiki, hay que tener en cuenta que está diseñada y optimizada para sitios web con mucho tráfico, por lo que, si el espacio en el disco o la memoria están limitados, quizá haya que buscar otra wikialternativa. También puede plantear problemas a la hora de restringir el acceso de lectura a ciertos contenidos y a otros, no.

## 2.3. Javadoc

Javadoc es una herramienta de Oracle para generar documentación técnica, normalmente, en formato HTML, a partir de archivos escritos en lenguaje Java. La herramienta Javadoc toma como referencia los comentarios incluidos en archivos de código fuente o en los paquetes para generar la documentación de manera automática.

Para que los comentarios se tengan en cuenta, deben aparecer justo antes de la declaración de una clase, de un campo, de un constructor o de un método. Los comentarios deben estar escritos en un formato establecido, que podemos simplificar así:

- » El comentario empieza con una línea con el delimitador `/**`.
- » Termina con otra línea, pero esta vez, con el delimitador `*/`.
- » El resto de las líneas del comentario empiezan con `*`.
- » El comentario tiene un primer bloque que contiene la descripción del elemento al que precede. Esta descripción puede contener una primera línea, que será incluida en la descripción corta, y otra, más larga, que se incluirá en el detalle.
- » Además, tiene un segundo bloque, separado del anterior con una línea que contiene un solo `*`, donde se definen una serie de etiquetas precedidas del carácter `@` y seguidas de una descripción.

» Cada línea del comentario debe tener, como máximo, 80 caracteres.

A continuación, se presenta una lista de algunas de las etiquetas que se pueden usar y el orden en el que deben aparecer.

ETIQUETA	EXPLICACIÓN
<code>@author</code>	Indica el autor de las clases e interfaces. Es obligatoria.
<code>@version</code>	Indica la versión de las clases e interfaces. Es obligatoria.
<code>@param</code>	Indica el nombre y describe los parámetros, en orden, del método o constructor. Es obligatorio para cada uno de los parámetros.
<code>@return</code>	Describe el valor devuelto por un método.
<code>@exception</code> <code>@throws</code>	Indica el nombre de una excepción que puede lanzar un método. Es obligatoria.
<code>@see</code>	Aconseja consultar algún otro recurso. Puede incluir enlaces.
<code>@deprecated</code>	Indica que el elemento ya no debería usarse.

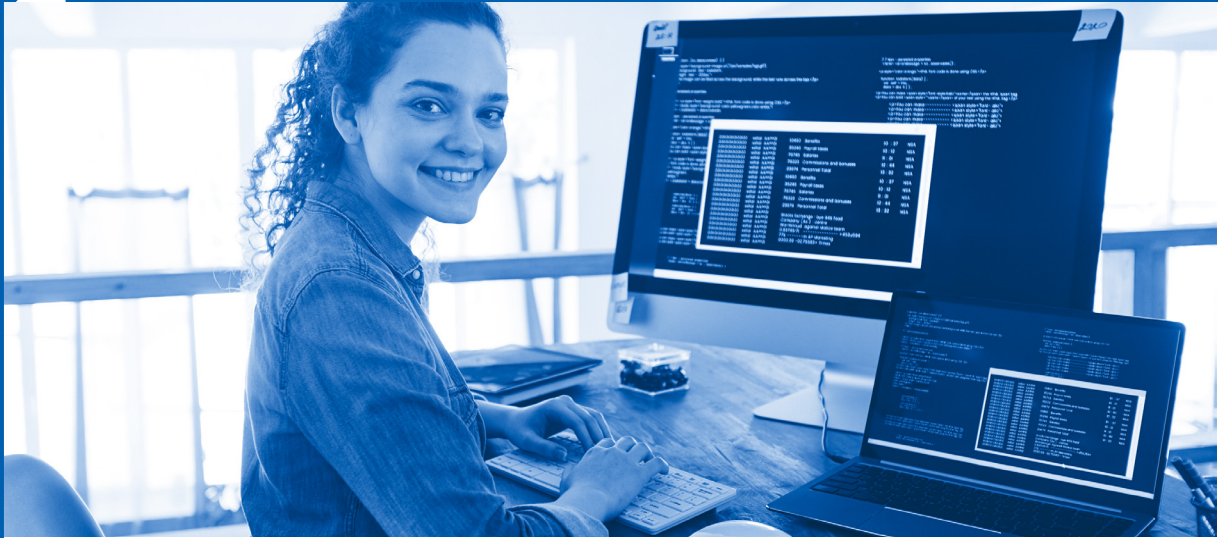
Tabla 3.

Este podría ser un comentario, compatible con Javadoc, de un método que recibe como argumento los datos de una persona y la da de alta en la base de datos como alumno o alumna.

```
/**
 * Alta de alumno(a) .
 * Da de alta a un alumno(a) en la base de datos.
 *
 * @param persona Datos del alumno(a) a dar de alta.
 * @throws IllegalArgumentException Si el argumento no es válido.
 */
```

Una vez que el código se encuentra debidamente comentado, se puede generar la documentación con el comando `javadoc` o con una herramienta IDE (*Integrated Development Environment*). Este tipo de herramientas, además de generar la documentación, usan los comentarios para ofrecer una ayuda interactiva mientras se programa.

## 2 CASO PRÁCTICO



### INTEGRACIÓN DE GIT CON GITHUB

#### Introducción

Una vez creado un repositorio local y después de haber practicado con los comandos básicos para tener un correcto control de versiones cada vez que se cambie algo en alguno de los archivos del proyecto, vas a integrar tu repositorio local con un repositorio remoto en GitHub. La idea es publicar tu repositorio en la nube para que otros usuarios puedan descargarlo y trabajar de forma local en sus equipos.

Por tanto, te crearás una cuenta de usuario en la herramienta GitHub, crearás un repositorio público y subirás tu repositorio local.

Antes de comenzar, prepara otra máquina con Git instalado y un usuario diferente al de la primera máquina. Trabajarás en esta segunda máquina simulando que eres otro usuario que ha clonado tu repositorio para trabajarlo de forma local en él.

#### Claves de resolución

1. Verificar que tu repositorio Git que has utilizado en las actividades anteriores está en funcionamiento.
2. Crear una cuenta de usuario en GitHub y crear un repositorio nuevo.
3. Crear un token de acceso personal.
4. Hacer push desde el repositorio local al de GitHub.
5. Utilizar una segunda máquina con un segundo usuario para clonar el repositorio de GitHub llamado proyecto y trabajarlo de forma local.

**Nota:** Para resolver este caso práctico extendido se ha utilizado la misma máquina virtual Ubuntu Desktop que en el Caso práctico y en el Ponte a prueba de este apartado. Además, se ha utilizado otra máquina igual y se ha configurado Git para simular el trabajo de un segundo usuario.

## SOLUCIÓN

1. Para verificar que tu repositorio Git funciona correctamente, puedes lanzar el comando `status`. Debe devolvarte algo similar a esto:

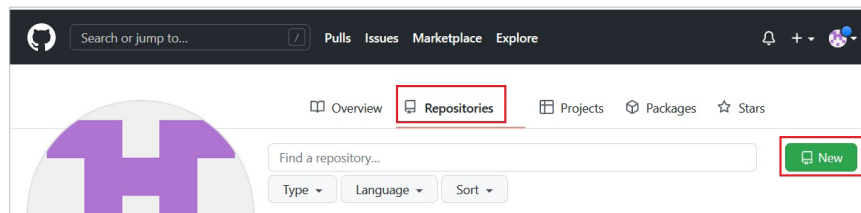
```
admin01@cliente:~/proyecto$ git status
```

En la rama master

nada para hacer commit, el árbol de trabajo está limpio

2. Para crear una cuenta en GitHub, entra en el sitio web <https://github.com/> y sigue las indicaciones para registrarte.

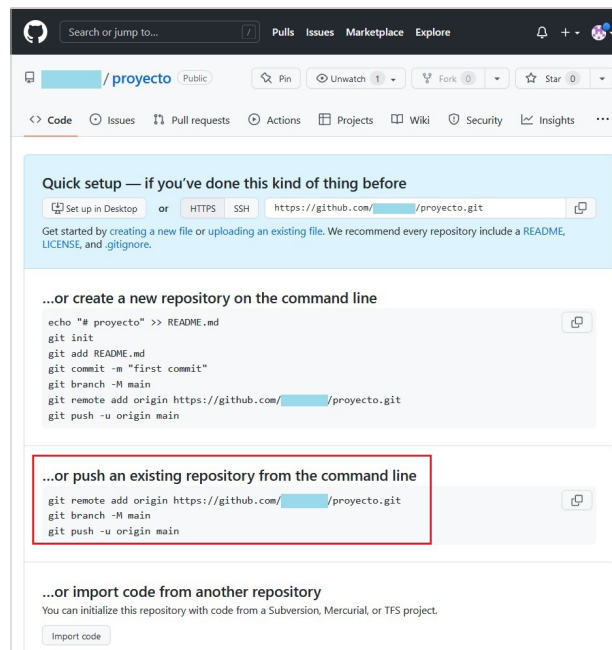
Para añadir un repositorio a tu cuenta de GitHub, debes entrar en el apartado *Repositories (Repositorios)* y hacer clic en el botón *New (Nuevo)*.



Aparecerá la pantalla para indicar el nombre del repositorio. En el ejemplo, el nombre es *proyecto*. Puedes dejar el resto de opciones tal como aparecen y hacer clic en el botón *Create repository (Crear repositorio)*. Puedes ver las opciones que aparecen en la siguiente captura de pantalla:

A screenshot of the GitHub 'Create repository' form. The 'Owner' field is set to a user icon, and the 'Repository name' field is set to 'proyecto' with a green checkmark. Below the name field, there is a suggestion: 'Great repository names are short and memorable. Need inspiration? How about [studious-octo-train](#)?'. The 'Description (optional)' field is empty. The 'Public' radio button is selected, with the text 'Anyone on the internet can see this repository. You choose who can commit.' below it. The 'Private' radio button is unselected, with the text 'You choose who can see and commit to this repository.' below it. The 'Initialize this repository with:' section has a note: 'Skip this step if you're importing an existing repository.' It includes a checkbox for 'Add a README file' with a link to 'Learn more.' and a dropdown for '.gitignore template' set to 'None'. The 'Choose a license' section has a note: 'A license tells others what they can and can't do with your code. [Learn more.](#)' and a dropdown for 'License' set to 'None'. At the bottom, there is a green 'Create repository' button and a note: 'You are creating a public repository in your personal account.'

Al crear el repositorio, se abre una pantalla con su información, la puedes ver en la siguiente captura de pantalla. Como el repositorio todavía está vacío, GitHub muestra las opciones que hay para incorporar información y una de ellas es mediante la orden *push*. No sigas las instrucciones de la captura ahora, lo harás en el punto 4.



**3.** Para crear un *token* de acceso personal:

- Entra en tu cuenta de GitHub y accede al apartado *Configuración* desde el desplegable que aparece al hacer clic en tu foto de perfil.
- Ve a la barra lateral izquierda, haz clic en *Ajustes de desarrollador* y, luego, haz clic en *Tokens de acceso personal*.
- Haz clic en el botón *Generar un nuevo token*.
- Escribe el nombre del *token*, indica su vida útil y marca la casilla de verificación *repo*. Por último, haz clic en el botón *Generar token*.
- Copia tu *token* en un sitio seguro.

**4.** Para hacer *push* desde el repositorio local, hay que ejecutar las órdenes que han aparecido en la captura de pantalla anterior:

```
admin01@cliente:~/proyecto$ git remote add origin https://github.com/primer_usuario/proyecto.git
admin01@cliente:~/proyecto$ git branch -M main
admin01@cliente:~/proyecto$ git push -u origin main
```

Para el ejemplo, se ha utilizado *primer_usuario* para indicar el nombre con el cual el usuario se ha registrado en GitHub. A continuación, se detalla qué hace cada comando:

» **git remote add origin [https://github.com/primer_usuario/proyecto.git](https://github.com/primer_usuario/proyecto.git)**: crea una conexión llamada *origin* con el repositorio remoto que hay en la URL [https://github.com/primer_usuario/proyecto.git](https://github.com/primer_usuario/proyecto.git).

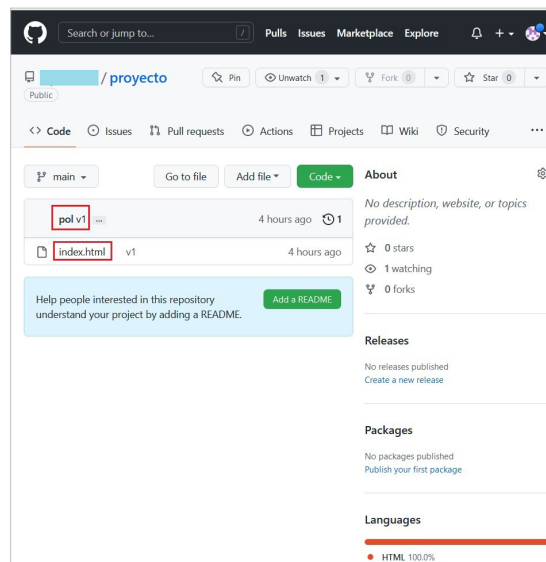
» **git branch -M main**: indica la rama del repositorio en la que se va a trabajar.

» **git push -u origin main**: hace la sincronización en el repositorio apuntado por *origin* en la rama *main*.

Al ejecutar la orden *push*, se pide usuario y contraseña de la cuenta de GitHub. En el campo *Contraseña* debes introducir el *token* que has creado en el paso anterior. Si todo va bien, deberás ver las siguientes líneas:

```
Username for 'https://github.com': primer_usuario
Password for 'https://primer_usuario@github.com':
Enumerando objetos: 3, listo.
Contando objetos: 100% (3/3), listo.
Escribiendo objetos: 100% (3/3), 206 bytes | 206.00 KiB/s, listo.
Total 3 (delta 0), reusado 0 (delta 0), pack-reusado 0
To https://github.com/primer_usuario/proyecto.git
* [new branch]      main -> main
Rama 'main' configurada para hacer seguimiento a la rama remota
'main' de 'origin'.
```

Si ahora recargas la página web del repositorio en GitHub, verás algo parecido a la siguiente captura de pantalla:

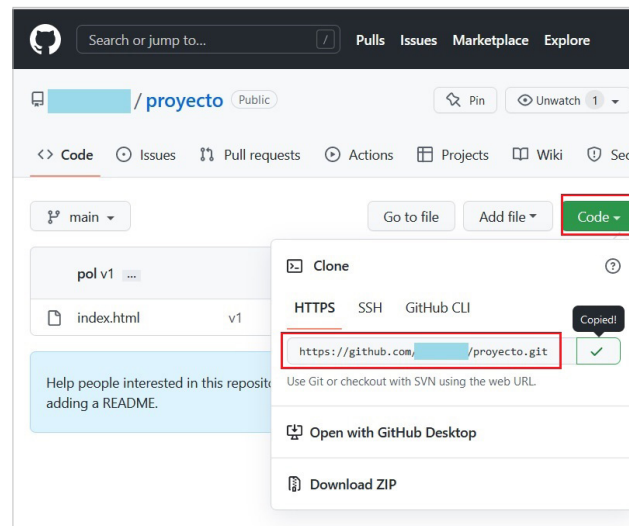


Puedes observar el nombre del usuario que utilizaste en tu repositorio local y la versión que indicaste cuando hiciste el *commit*. Como puedes ver, el archivo que aparece es **index.html**, el mismo del repositorio local de tu máquina de trabajo.

5. Ahora vas a trabajar como si fueras el segundo usuario, desde la segunda máquina local. Asegúrate que estás en el directorio de inicio.

```
admin01@cliente2:~$ cd ~
```

Para clonar el repositorio remoto de GitHub al local, has de entrar en el sitio web de GitHub y localizarlo. No hace falta que el segundo usuario esté registrado en GitHub, ya que el primer usuario ha dejado el repositorio de forma pública. Para localizar el repositorio en GitHub, puedes buscar por nombre de usuario y ver sus repositorios públicos. Cuando localices el repositorio llamado *proyecto*, entra en él, haz clic en el botón *Code (Código)* y copia el enlace. A continuación, tienes una captura de pantalla:



Desde el terminal de la máquina local del segundo usuario, ejecuta:

```
admin01@cliente2:~$ git clone https://github.com/primer_usuario/
proyecto.git
Clonando en 'proyecto'...
remote: Enumerating objects: 3, done.
remote: Counting objects: 100% (3/3), done.
remote: Total 3 (delta 0), reused 3 (delta 0), pack-reused 0
Recibiendo objetos: 100% (3/3), listo.
```

Si te fijas, la URL de la primera instrucción es la que has copiado desde GitHub. Comprueba que se ha creado el directorio **proyecto** y, después, mira que dentro tenga el directorio oculto **.git** y el archivo **index.html**.

```
admin01@cliente2:~$ cd proyecto
admin01@cliente2:~/proyecto$ ls -a
.  ..  .git  index.html
```

Haz varios cambios en el archivo **index.html** y guárdalos en el repositorio local del segundo usuario. Para ello, puedes ejecutar las instrucciones siguientes:

```
admin01@cliente2:~/proyecto$ nano index.html
admin01@cliente2:~/proyecto$ git add index.html
admin01@cliente2:~/proyecto$ git commit -m v2
```

## 03 Control de versiones

Como se ha comentado en la introducción de esta unidad, llevar un control de versiones es clave a la hora de desarrollar proyectos y más aún si en su realización interviene más de un programador. Ya se han comentado las ventajas que aporta el control de versiones. En este apartado se tratará su funcionamiento y se explicarán algunas herramientas que existen actualmente a disposición de cualquier desarrollador.

Con la finalidad de poder explicar correctamente el funcionamiento de los sistemas de control de versiones, se definirán algunos conceptos:

<b>Repositorio:</b>	Es el espacio de almacenamiento donde se guardan los archivos con el código fuente de la aplicación y también sus metadatos o información asociada. Básicamente, la idea es que este repositorio sea siempre la última versión válida del programa y cualquier desarrollador pueda incorporar los cambios que ha realizado y obtener los que han incorporado otros integrantes del grupo de trabajo. El sistema se encarga de controlar que los cambios efectuados por los programadores no se interfieran entre ellos, por ejemplo, en el caso de que dos programadores hayan efectuado diferentes cambios en un mismo archivo. Este espacio de almacenamiento puede estar en una máquina local si el proyecto solamente se desarrolla por un único programador, en un servidor ubicado en una intranet si el desarrollo de la aplicación se realiza desde una red de área local empresarial o, por último, en un espacio en la nube. Al repositorio deben poder acceder todos los programadores involucrados en el proyecto y cada uno de ellos trabaja en una copia local del mismo.
<b>Cambio</b>	Básicamente, un cambio es una modificación sobre alguno de los archivos del repositorio.
<b>Conflicto</b>	Si, entre que un usuario copia un archivo del repositorio y lo actualiza, otro usuario aplica sus propios cambios sobre ese mismo archivo, se produce un conflicto; entonces, el sistema de control de versiones lo detecta y avisa a los usuarios para que lo resuelvan y decidan qué versión del archivo es la que debe prevalecer.
<b>Revisión</b>	Una revisión es una versión que se identifica con un número.
<b>Ramas</b>	Un sistema de control de versiones debe permitir trabajar en paralelo a partir del desarrollo principal del proyecto, bien para hacer pruebas o porque en el proyecto participan varios programadores y cada uno de ellos desarrolla diferentes funcionalidades de la aplicación. Para poder trabajar de esta forma, los sistemas de control de versiones disponen de <b>ramas</b> . Se puede ver el repositorio como un árbol en el que se llevan a cabo distintos flujos de trabajo en paralelo sin interferirse entre ellos ni provocar conflictos. Cada uno de estos flujos de trabajo es una rama. Es como si, a partir de un momento concreto, el proyecto se desarrollara de dos formas distintas con sus correspondientes revisiones.

Algunas de las herramientas de control de versiones más conocidas son las siguientes:

<b>Apache Subversion (SVN)</b>	utiliza el modelo de repositorio cliente-servidor. Mantiene versiones de los directorios de trabajo y de las acciones sobre los archivos, como borrados, cambios de nombre, etc.
<b>CVS</b>	su modelo de repositorio también es cliente-servidor. Permite que distintos programadores trabajen en un mismo proyecto al mismo tiempo y solo la resolución de conflictos necesita tratarse manualmente.
<b>Git</b>	el modelo de repositorio que utiliza es el distribuido, y puede trabajar con los protocolos HTTP, FTP y SSH. Es el sistema de control de versiones que se desarrollará a continuación por ser uno de los más utilizados.

## 3.1. Git

Git es un sistema de control de versiones que se publica bajo la Licencia Pública General GNU versión 2.0 (GPLv2). Está disponible para los sistemas operativos GNU/Linux, macOS y Windows.

Sus tiempos de ejecución son muy rápidos, ya que, por un lado, al estar escrito en C, su velocidad es mayor que si estuviera escrito en otro lenguaje de alto nivel. Por otro lado, el hecho de que fuera creado para gestionar el desarrollo del núcleo (*kernel*) de Linux hizo que tuviera que manejar desde el principio grandes repositorios, por lo que también debió tener en cuenta la velocidad.

Tiene un modelo de ramificación muy flexible que permite:

- » tener ramas dedicadas a diferentes tareas (por ejemplo, una rama con el código fuente, que irá a producción, y otra para pruebas, que no afecta al proyecto principal);
- » fusionar ramas entre ellas o eliminarlas cuando ya no son necesarias.

Es un sistema distribuido en el que se dispone de un repositorio completo en el servidor y cada usuario tiene una copia para trabajar en local. La información viaja cifrada en ambos sentidos. Tras su instalación, requiere que se indiquen algunos parámetros de configuración. Esta configuración inicial puede establecerse:

- » **en el sistema:** se aplicará para todos los usuarios;
- » **en cada usuario:** se aplicará solamente para un usuario determinado;
- » **en el repositorio:** se aplicará solamente sobre un repositorio determinado.

Elegir una configuración u otra dependerá de las características del proyecto y del equipo de desarrolladores que participan en él.

## A. Forma de trabajo

Git utiliza las siguientes tres áreas de trabajo:

- » **Directorio de trabajo (*working directory*)**: en él hay una copia de los archivos del repositorio para poder modificarlos.
- » **Área de preparación (*staging area*), o índice**: es un área intermedia donde se almacenan los cambios que se realizarán sobre el repositorio la próxima vez que se lance una orden de confirmación. Así, aunque se hayan modificado varios archivos en el directorio de trabajo, solamente los que estén en el área de preparación se enviarán al ejecutar la instrucción de confirmación.
- » **Directorio de Git (*Git directory*), o repositorio**: contiene el proyecto y sus metadatos, además de un registro histórico de los diferentes cambios efectuados a lo largo de la vida útil del proyecto. HEAD es el puntero de la última confirmación o de la última versión.

Por tanto, los archivos del proyecto podrán estar en una de esas tres áreas o, más bien, tener alguno de los siguientes estados que se corresponden con ellas:

- » **No modificado**: un archivo no modificado se encuentra en el directorio de trabajo y está en este estado cuando, después de obtenerlo del repositorio, todavía no ha sufrido ningún cambio.
- » **Modificado (*modified*)**: un archivo modificado se encuentra en el directorio de trabajo y está en este estado cuando, después de obtenerlo del repositorio, ha sufrido cambios, pero todavía no se ha ejecutado sobre él la orden de preparación.
- » **Preparado (*staged*)**: un archivo marcado para confirmación se encuentra en el área de preparación y está en este estado si, después de obtenerlo del repositorio, ha sufrido cambios y ya se ha ejecutado sobre él la orden de preparación.
- » **Confirmado (*committed*)**: un archivo confirmado se encuentra ya registrado en el repositorio.

## B. Comandos

De entre todos los comandos que dispone Git, a continuación se nombrarán algunos de los más utilizados agrupados según su función:

**CREAR PROYECTOS**

<b>init</b>	Crea un repositorio Git vacío en el que solamente están los subdirectorios para objetos, referencias y archivos de plantilla. También crea una rama inicial sin ninguna confirmación.
<b>clone</b>	Clona un repositorio en un directorio nuevo y crea una rama de seguimiento remoto para cada una de las ramas del repositorio clonado.

**INSTANTÁNEAS BÁSICAS**

<b>add</b>	Añade el contenido de un archivo al área de preparación. Este archivo se tendrá en cuenta al ejecutar la próxima confirmación. El comando se puede ejecutar varias veces antes de ejecutar una confirmación.
<b>status</b>	Muestra el estado del área de preparación, es decir, indica los archivos y carpetas que tienen diferencias entre el área de preparación y la confirmación actual.
<b>diff</b>	Muestra diferencias entre diferentes confirmaciones o entre una confirmación y el área de preparación.
<b>commit</b>	Confirma los cambios en el repositorio. Es decir, crea una nueva versión con el contenido actual del área de preparación y guarda un mensaje con los cambios que se han hecho en el registro de actividad. Luego, marca todos los archivos que estaban en estado "preparado" a estado "no modificado".
<b>notes</b>	Agrega, elimina o lee notas asociadas a objetos sin alterarlos.
<b>restore</b>	Restaura archivos del directorio de trabajo a partir de una fuente de restauración. Si un archivo no existe en la fuente de restauración, se eliminará del directorio de trabajo.
<b>reset</b>	Restablece HEAD al estado especificado, es decir a un estado de confirmación diferente al que tenía.
<b>rm</b>	Elimina archivos del directorio de trabajo y del área de preparación.
<b>mv</b>	Mueve o renombra un directorio o un archivo.

### RAMIFICACIÓN Y FUSIÓN

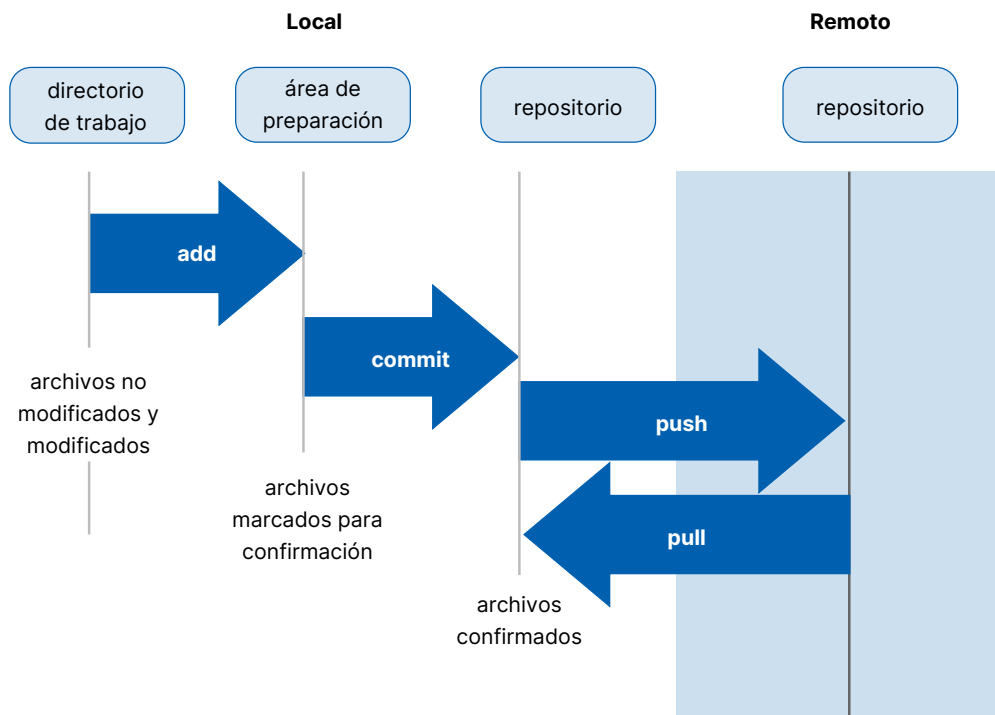
<b>branch</b>	Enumera, crea o elimina ramas.
<b>checkout</b>	Cambia de rama o restaura los archivos del directorio de trabajo, es decir, actualiza los archivos en el directorio de trabajo para que coincidan con la versión actual del repositorio.
<b>switch</b>	Cambia de rama. El directorio de trabajo y el área de preparación se actualizan para que coincidan con la nueva rama. Todas las confirmaciones nuevas se agregan a la nueva rama.
<b>merge</b>	Incorpora cambios de las confirmaciones indicadas a la rama actual. Es útil para incorporar cambios de una rama a otra.
<b>mergetool</b>	Ejecuta herramientas de resolución de conflictos para poder resolverlos. Normalmente, se suele ejecutar después del comando <b>git merge</b> .
<b>log</b>	Muestra los registros de confirmación. Por defecto, las confirmaciones aparecen en orden cronológico inverso.
<b>stash</b>	Guarda los cambios en un directorio de trabajo de pruebas. Se utiliza cuando se quieren realizar pruebas con la posibilidad de volver a un directorio de trabajo anterior.
<b>tag</b>	Crea, enumera, elimina o verifica una etiqueta en un punto concreto del historial del repositorio.
<b>worktree</b>	Administra múltiples ramas de trabajo asociadas a un repositorio.

### COMPARTIR Y ACTUALIZAR PROYECTOS

<b>fetch</b>	Descarga objetos y referencias de otro repositorio.
<b>pull</b>	Actualiza el repositorio local desde el remoto.
<b>push</b>	Actualiza el repositorio remoto desde el local.

Tabla 4.

En el siguiente esquema, se pueden ver de forma conjunta las áreas de trabajo y algunos comandos que indican el flujo de trabajo y cómo los archivos pueden pasar de un estado a otro en función del comando que se les aplique:



Esquema 1.

## 3.2. GitHub

En el BOE-A-2023-13221 DAW, dentro de los criterios de evaluación están la creación y los tipos de usuarios y grupos.

GitHub, la plataforma líder en desarrollo colaborativo de software, es mucho más que un simple repositorio de código. Con millones de usuarios en todo el mundo, su impacto en la comunidad de desarrollo de software es innegable. La diversidad de usuarios en GitHub es tan amplia como las posibilidades que ofrece la programación misma. Desde estudiantes de informática hasta empresas multinacionales, cada uno encuentra su lugar en esta plataforma. Vamos a explorar los diferentes tipos de usuarios de GitHub y cómo aprovechan esta herramienta según sus necesidades y objetivos.

### 1. Estudiantes de Grado Superior de Informática

Para los estudiantes de informática, GitHub es más que un lugar para almacenar proyectos. Es una herramienta educativa invaluable que les permite aprender sobre control de versiones, colaboración en equipo y prácticas de desarrollo de software en un entorno real. Utilizan GitHub para compartir proyectos con compañeros de clase, colaborar en asignaciones grupales y, lo más importante, construir un portafolio digital que muestre sus habilidades a posibles empleadores.

## 2. Desarrolladores Profesionales

Los desarrolladores profesionales encuentran en GitHub un espacio para colaborar en proyectos de código abierto, contribuir a bibliotecas y herramientas populares, y compartir su experiencia con la comunidad. Además, utilizan GitHub como parte de su flujo de trabajo diario, gestionando proyectos privados de equipos de desarrollo, realizando seguimiento de problemas y solicitando y revisando cambios mediante solicitudes de extracción (pull requests).

## 3. Empresas y Organizaciones

Para las empresas y organizaciones, GitHub ofrece soluciones empresariales que van más allá del alojamiento de código. Utilizan GitHub para gestionar proyectos, automatizar flujos de trabajo con acciones (actions) y mejorar la colaboración entre equipos distribuidos geográficamente. Además, muchas empresas utilizan GitHub como parte de su estrategia de reclutamiento, evaluando el código y la actividad de los candidatos potenciales en la plataforma.

## 4. Mantenedores de Proyectos Open Source

Los mantenedores de proyectos de código abierto son una parte fundamental de la comunidad GitHub. Son responsables de mantener y mejorar bibliotecas, marcos de trabajo y herramientas utilizadas por millones de desarrolladores en todo el mundo. Utilizan GitHub para gestionar problemas, revisar y fusionar contribuciones de la comunidad, y garantizar la calidad y estabilidad de sus proyectos.

## 5. Entusiastas y Aficionados

Además de los usuarios profesionales, GitHub también atrae a entusiastas y aficionados que comparten un interés común en la programación y la tecnología. Estos usuarios utilizan GitHub para explorar proyectos interesantes, aprender nuevas tecnologías y contribuir de forma ocasional a proyectos de código abierto que les apasionan.

En resumen, GitHub es un ecosistema diverso que reúne a una amplia gama de usuarios, desde estudiantes de informática hasta empresas multinacionales. Cada uno encuentra valor en la plataforma de acuerdo con sus necesidades y objetivos individuales. Ya sea para aprender, colaborar, construir o compartir, GitHub continúa siendo un recurso invaluable para la comunidad de desarrollo de software en todo el mundo.

GitHub también está pensado para trabajar con integración continua (*continuous integration*) y entrega continua (*continuous delivery*), en adelante, CI/CD, cuya finalidad es conseguir que los tiempos de desarrollo e implantación de una aplicación sean menores.

**CI:** consiste en que los cambios de programación se prueban e integran automáticamente en un repositorio compartido. En la programación tradicional, los cambios de código realizados por varios programadores se fusionan todos juntos en un momento del tiempo

próximo al despliegue de la aplicación y en ese momento es cuando surgen errores o conflictos que deben resolverse y, por tanto, el momento de desplegar la aplicación se retrasa. Si las pruebas se realizan cada vez que un programador introduce un cambio y se verifica que no hay fallos derivados de esos cambios, se ahorra mucho más tiempo que si se hacen las pruebas de todas las modificaciones hechas por todos los programadores en un mismo momento al final del trabajo.

**CD:** consiste en publicar nuevas versiones del producto en cuanto estén disponibles, sin esperar a tener grandes cambios, de manera que las mejoras están disponibles casi inmediatamente para los usuarios. En este tipo de despliegue, es muy importante tener un entorno de pruebas para verificar que las modificaciones introducidas por los desarrolladores son correctas y que la aplicación funciona de forma estable.

Los repositorios alojados en GitHub pueden utilizar la herramienta *GitHub Actions* para automatizar las acciones necesarias para detectar conflictos y realizar pruebas de funcionamiento cada vez que se aplica un cambio en el repositorio.

## A. Cuentas de GitHub

GitHub se ha convertido en una herramienta fundamental para desarrolladores de software en todo el mundo, permitiendo la colaboración, el seguimiento de versiones y la gestión de proyectos de manera eficiente. Sin embargo, para aquellos que recién comienzan su viaje en el mundo del desarrollo de software, entender las diferentes opciones de cuentas en GitHub puede resultar un tanto confuso. A continuación, exploraremos los tipos de cuentas de GitHub más comunes, así como ejemplos de cómo se pueden utilizar en proyectos reales.

### 1. Cuenta Gratuita

La cuenta gratuita en GitHub es la opción más básica y accesible para cualquier persona interesada en la plataforma. Con esta cuenta, los usuarios pueden crear repositorios públicos ilimitados y colaborar con otros desarrolladores en proyectos de código abierto. Sin embargo, las funcionalidades pueden ser limitadas en comparación con las cuentas de pago.

Por ejemplo, si estás trabajando en un proyecto para crear un sitio web, puedes crear un repositorio público para tu proyecto y colaborar con tus compañeros de clase para desarrollar el código. Además, puedes aprovechar las características de seguimiento de problemas y solicitudes de extracción para gestionar las tareas y revisar los cambios de código.

### 2. Cuenta Pro

La cuenta Pro de GitHub está diseñada para desarrolladores individuales y equipos que requieren características avanzadas y un mayor control sobre sus repositorios. Con esta cuenta, los usuarios tienen acceso a repositorios privados ilimitados, integración con herramientas de desarrollo populares y soporte prioritario.

Por ejemplo, si estás trabajando en un proyecto de software comercial con un equipo de desarrolladores, con una cuenta Pro de GitHub puedes crear repositorios privados para mantener el código seguro y protegido. Además, puedes integrar herramientas de integración continua, como Travis CI o CircleCI, para automatizar pruebas y despliegues, lo que garantiza que el código se construya y entregue de manera confiable en cada cambio.

### 3. Cuenta de Equipo

La cuenta de Equipo de GitHub está diseñada para organizaciones y empresas que necesitan colaborar en proyectos a gran escala. Esta cuenta ofrece características adicionales, como administración de acceso granular, revisión de código avanzada y herramientas de seguridad mejoradas.

Por ejemplo, si trabajas en una empresa de desarrollo de software con múltiples equipos, con una cuenta de Equipo de GitHub puedes crear organizaciones para cada equipo y gestionar el acceso a los repositorios de manera granular. Además, puedes utilizar características como la revisión de código y los controles de seguridad avanzados para garantizar la calidad y la seguridad del código en todos los proyectos.

En resumen, GitHub ofrece una variedad de opciones de cuentas para adaptarse a las necesidades de desarrolladores individuales, equipos y organizaciones. Ya sea que estés empezando con una cuenta gratuita o administrando proyectos a gran escala con una cuenta de equipo, GitHub proporciona las herramientas necesarias para colaborar de manera efectiva y construir software de calidad. Los estudiantes pueden dar un paso más en su viaje en el mundo del desarrollo de software, aprovechando al máximo esta poderosa plataforma de desarrollo colaborativo.



#### VÍDEOLAB

Para consultar el siguiente vídeo sobre documentación técnica en Java con IntelliJ y Javadoc, escanea el código QR o [pulsa aquí](#).

**UAX** FP